geometry-gen

Teaching a small language model to turn natural-language
Olympiad geometry problems into renderable GeoGebra constructions

Project report

June 10, 2026

Abstract

geometry-gen is an end-to-end experiment that fine-tunes a small, laptop-runnable language model to translate a natural-language plane-geometry problem into a machine-executable *GeoGebra construction*: an ordered list of construction steps that draws an interactive, draggable diagram of the problem. The defining idea of the project is that a *real geometry engine is used as an automatic correctness oracle*. Every construction — both the training data and the model's output — is rendered in a headless GeoGebra engine and is accepted only if every step produces an object that exists; an optional second layer probes the resulting coordinates and verifies that the stated theorem holds numerically. This oracle is what makes the dataset trustworthy and the evaluation faithful to production behaviour. We curate 82 gold problems, programmatically grow the corpus to 232 verified problems, QLoRA-fine-tune **Qwen2.5-Coder-3B-Instruct** on the result, and evaluate *functionally* — by rendering the model's predictions. On a held-out set of 31 problems the fine-tuned model produces schema-valid JSON 100% of the time and a construction that renders as a valid, non-degenerate figure 61% of the time before any repair, with a generate–validate–repair loop available to recover further cases.

Contents

1 Overview and motivation

A geometry problem is usually accompanied by a figure. Producing a *correct, interactive* figure from the prose is itself hard: the diagram must realise the stated constraints structurally (a midpoint must be a real midpoint, an incircle must actually touch the sides), it must be non-degenerate, and it must remain manipulable. geometry-gen treats this as a **constrained code-generation** task: the target "code" is a JSON list of native GeoGebra commands.

The central engineering principle is the use of GeoGebra itself as a **verifier in the loop** at every stage:

1. to *certify* the 82 hand-curated seed problems;
2. to *gate* every machine-generated problem added to the corpus;
3. to choose *training labels* (which objects are shown vs. hidden);
4. to *filter* the model's output at inference time; and
5. to *score* the model with a functional metric that mirrors production.

Because the model only needs to be good enough that a short validate-and-repair loop reaches a renderable figure, a 3B-parameter model running in 4-bit on a 6 GB GPU (or a laptop CPU) suffices.

1.1 System at a glance

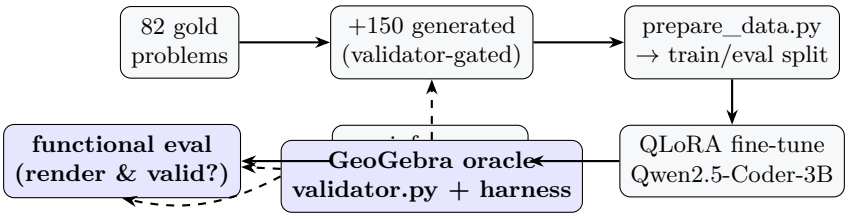

Figure 1: The same GeoGebra oracle (dashed) gates data generation and scores the model.

2 Data representation

2.1 The construction schema

Each problem is a single JSON object (one line of a JSONL file) with a `problem_statement` and an ordered `construction_steps` array. Each step has five fields:

field	meaning
`name`	unique identifier; later steps reference it. Steps execute top-to-bottom, so a command may only use names defined earlier (a constructible/topological order).
`type`	`"final"` (shown) or `"construction"` (hidden helper / scaffolding).
`class`	`point` | `line` | `segment` | `ray` | `circle` | `conic` | `number` | `angle` | `vector` | `list` | `locus` | `polygon`.
`label`	optional display caption (e.g. Γ, A').
`command`	native GeoGebra syntax: a coordinate literal `(0, 2)` for a free point, or a command such as `Circle(O, 2)`, `Midpoint(B, C)`, `Intersect(Γ, c, 2)`, `Reflect(H, O)`, `(A + B + C) / 3`.

A worked example (the median construction, the centroid theorem):

```
{"id": 1,
 "problem_statement": "In triangle ABC let M, N, P be the midpoints of
   BC, CA, AB. Prove the medians AM, BN, CP are concurrent (centroid G) ...",
 "construction_steps": [
   {"name":"A","type":"final","class":"point","label":"A","command":"(-4, -2)"},
   {"name":"B","type":"final","class":"point","label":"B","command":"(5, -1)"},
   {"name":"C","type":"final","class":"point","label":"C","command":"(1, 5)"},
   {"name":"M","type":"final","class":"point","label":"M","command":"Midpoint(B, C)"},
   {"name":"N","type":"final","class":"point","label":"N","command":"Midpoint(C, A)"},
   {"name":"P","type":"final","class":"point","label":"P","command":"Midpoint(A, B)"},
   {"name":"median_A","type":"final","class":"segment","command":"Segment(A, M)"},
   ...
   {"name":"G","type":"final","class":"point","label":"G","command":"(A + B + C) / 3"}]}
```

2.2 Free points on a path: the parameterisation trick

A "free point on circle ω" is written `Point(omega)`. If several such points share one path, GeoGebra collapses them all onto the path's anchor. The renderer/harness therefore auto-injects a parameter: the i-th of k points sharing a path is placed at $t = (i+1)/(k+1)$, or (for robustness testing) at a random value stratified into the slice $[\,i/k, (i+1)/k\,]$. Crucially the point is created as a bare `Point(path)` and then *nudged* to t with `SetValue`, so it stays *draggable* — writing `Point(path, t)` directly would pin it. Points on infinite lines are excluded from this treatment because the parameterised form there can become non-draggable.

2.3 Datasets actually shipped

file	problems	steps (min/med/max)	role
`geometry_problems.jsonl`	82	8 / 19 / 34	gold seed + style anchor
`datagen/generated_problems.jsonl`	150	6 / 13 / 23	machine-grown corpus
total training pool	232	—	merged for fine-tuning

Optional semantic checks. Some problems carry a top-level `checks` array asserting the theorem they illustrate, drawn from a rich vocabulary: `collinear`, `concyclic`, `concurrent`, `parallel`, `perpendicular`, `equal_len`, `on_circle`, `midpoint`, `tangent_circles`, `equal_area`, `len_product_equal` (power of a point), `angle_equal`, `angle_double` (inscribed angle), `ptolemy`, and more. These are verified numerically (Section 3).

3 The GeoGebra oracle: renderer, harness, validator

3.1 Renderer (`renderer.html`)

The human-facing tool: paste one problem's JSON, click **Render**, and a full interactive GeoGebra "classic" applet (toolbar, input bar, zoom, right-click properties) draws the figure with the statement shown above it. Key behaviours: command success is judged by `exists(name)` *after* `evalCommand` (GeoGebra's return value is unreliable); points/circles get labels but lines/segments/rays never do; `construction` objects are hidden unless a checkbox reveals them; and `fitView()` forces a 1:1 aspect ratio so circles render as circles.

3.2 Headless harness (`test_harness.html`)

The UI-less twin that mirrors the renderer's logic exactly and is driven from Python via Playwright/Chromium. It exposes `runSteps` (failures + coordinates), `probeFull` (*full-precision* coordinates for theorem checking), `runProblemRandom` (random-t build, failure count), and `renderLikeApp` (drag-test that points stay manipulable). It signals readiness via `window.__ggbReady`.

3.3 Validator and the robustness principle (`validator.py`)

`GeoGebraValidator` packages the oracle as a reusable context manager (localhost server + headless browser + harness). A construction can render cleanly for one lucky value of t; therefore the validator renders **several times with fresh random** t and accepts only if *every* trial is clean:

$$\text{accept} \iff \forall\,\text{trial} \in \{1,\dots,T\} : \#\{\text{steps that failed to produce an object}\} = 0.$$

The same class is reused for data generation, inference filtering, the repair loop, and evaluation — so the gate is identical everywhere.

3.4 Numeric theorem checks (`checks.py`)

Renderability is necessary but not sufficient: a *wrong intersection-index* pick can render fine yet be geometrically wrong. `checks.py` verifies the stated theorem against the full-precision probed coordinates with a relative tolerance $\tau = 10^{-6} \cdot (\text{figure scale})$. The key observation: a *true* theorem holds for all positions of the free points, whereas a wrong construction is off by $O(1)$, so τ cleanly separates them.

4 Data-generation pipeline (`datagen/`)

The aim is to grow the 82 seed problems into hundreds *without ever admitting a problem that fails to render.*

4.1 LLM generator (`generate.py`)

Claude (`claude-opus-4-8`, adaptive thinking) proposes one problem per call. Design highlights:

- **Few-shot anchoring:** a fixed, diverse set of 10 gold problems fixes schema and style.
- **Rules block:** encodes hard-won GeoGebra pitfalls (uppercase point names or a coordinate becomes a vector; `gamma` is reserved; incircle/circumcircle/center idioms; correct intersection index; tangents-from-a-point via a `{Tangent(...)}` list; no free point on an infinite line).
- **Prompt caching:** the ∼6k-token schema+rules+examples prefix sits in a cached system block; only a per-call theme and random nonce vary (after the cache point), so repeated calls re-read the prefix at ∼0.1× cost.

- **Validator-gated with one repair attempt:** each candidate is run through `validate_robust`; on failure the failing steps are fed back once for a correction, which is re-validated. Only survivors are written; generated ids are marked ≥ 1000.

4.2 No-API path and filters

`validate_candidates.py`
validates directly-authored candidates through two gates — (1) renderability, (2) if `checks` are present, numeric theorem truth — appending survivors, logging rejects with the failing step.

`post_filter.py`
the *inference-time* gate. A new problem has no ground-truth checks, so this applies generic degeneracy heuristics (points at infinity / non-finite; whole figure collapsed; too many coincident named points) and accepts by majority vote over trials (keep $\geq 60\%$ clean). Includes lenient JSON repair for small-model syntax slips.

`visibility.py`
the "show everything the statement names" rule: promotes statement-referenced `construction` steps to `final`; used both to post-process inference output and to relabel training data so the model *learns* the distinction.

`verify_dataset.py`
read-only audit that re-renders every problem and re-checks every assertion from scratch.

`selftest.py`, `post_filter_selftest.py`
exercise the plumbing with no API key.

4.3 Measured data quality

Every shipped problem was re-audited with the oracle for this report:

dataset	problems	render-clean	trials
gold (`geometry_problems.jsonl`)	82	82/82 (100%)	8 (all clean)
generated (`generated_problems.jsonl`)	150	150/150 (100%)	5 (all clean)

Every problem in the corpus renders robustly across all random-t trials — the dataset is, by construction, 100% valid.

5 Fine-tuning pipeline (`finetune/`)

5.1 Base model and hardware

Base model	`unsloth/Qwen2.5-Coder-3B-Instruct-bnb-4bit` (4-bit, $\sim$2 GB)
Why	code-specialised instruct model; largest that fits the target GPU
Hardware	RTX 3050, 6 GB VRAM (fallback to 1.5B documented)
Template	Qwen ChatML (`<|im_start|>role ...<|im_end|>`)
Toolchain	Unsloth + TRL `SFTTrainer` + PEFT/bitsandbytes

5.2 Corpus preparation (`prepare_data.py`)

The 232 problems are relabelled with the visibility rule, converted to three-message chat examples (system $\rightarrow$ schema instructions; user $\rightarrow$ problem statement; assistant $\rightarrow$ the construction JSON), and split **deterministically and stratified by source** (`seed=42`, 13% eval) so the held-out set contains both gold and generated problems:

$$\text{train} = 201, \qquad \text{eval} = 31 \ (= 11 \text{ gold} + 20 \text{ generated}).$$

Three files result: `train.jsonl`, `eval.jsonl` (chat format), and `eval_problems.jsonl` (raw statements for functional eval).

5.3 Training method: QLoRA

The 3B base is frozen and quantised to 4-bit; small **LoRA** adapter matrices are trained on top. Configuration:

LoRA rank r	16	max sequence length	1536
LoRA α	16	epochs	4
LoRA dropout	0	learning rate	2×10^{-4}, cosine
target modules	all attn + MLP proj.	warmup	5%
per-device batch	1	optimiser	8-bit AdamW
grad. accumulation	8 (eff. batch 8)	weight decay	0.01
precision	bf16/fp16 (auto)	seed	42

Target modules are `q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj`; Unsloth gradient checkpointing keeps activations within 6 GB.

5.4 Loss function and minimisation

The objective is the standard **autoregressive cross-entropy** (next-token) loss, computed **only over the assistant's JSON** — the prompt is masked out. For a target token sequence $y_1, \ldots, y_N$ with the model's predictive distribution p_θ, the per-example loss is

$$\mathcal{L}(\theta) \;=\; -\frac{1}{|\mathcal{R}|} \sum_{t \in \mathcal{R}} \log p_\theta(y_t \,|\, y_{<t}) \,, \qquad \mathcal{R} = \{\, t : y_t \text{ is part of the assistant response} \,\},$$

where every token *before* the assistant turn (the system prompt and the user's problem statement) has its label set to the ignore index (-100) and contributes nothing to $\mathcal{L}$. This is implemented with Unsloth's `train_on_responses_only` keyed on the ChatML markers `<|im_start|>user` and `<|im_start|>assistant`. The model is thus trained purely to *emit the construction*, not to reproduce the prompt.

How it is minimised. Gradients $\nabla_\theta \mathcal{L}$ flow *only into the LoRA adapter weights* (the quantised base is frozen). They are accumulated over 8 micro-batches and then applied by the **8-bit AdamW** optimiser; the learning rate follows a **cosine decay** from 2×10^{-4} after a 5% linear warmup, across 4 epochs of the 201 examples. Minimising $\mathcal{L}$ therefore means adjusting the small low-rank matrices so the model places higher probability on the correct construction-step tokens.

Why no in-loop validation loss. A full evaluation forward materialises full-vocabulary fp32 logits (∼2 GB) and would exhaust the 6 GB card, so `eval_strategy="no"`. Model quality is judged *functionally* instead (Section 6): does the generated construction actually render? Checkpoints are saved each epoch; final LoRA adapters land in `qwen25coder3b-geom-lora/`.

5.5 Inference, repair, and serving

`infer.py`
greedy decoding (`max_new_tokens`= 1280) over the held-out set → `predictions.jsonl`.

`repair_loop.py`
generate → validate → repair: failing constructions get the error fed back (up to 2 retries), re-validated; loads the model at 8k context because repair prompts are long. Reports accept rate before vs. after repair.

`eval_predictions.py`
the functional metric (Section 6), run where GeoGebra is available.

`export_gguf.py` / `geom.Modelfile` / `predict_local.py`
: merge LoRA → a single 4-bit `q4_k_m` GGUF (∼2 GB) served by Ollama with JSON-grammar-constrained decoding, so the whole system runs on a laptop CPU with no GPU.

6 Results

The metric that matters is **functional**: take the model's prediction for each held-out problem and ask whether it (a) parses as schema-valid JSON and (b) renders as a valid, non-degenerate figure under the production `post_filter` gate (7 random-t trials). Measured on the 31 held-out problems:

metric	**count**	**rate**
parseable schema JSON	31/31	100%
renders & valid (before repair)	19/31	61%
renders & valid | parseable	19/31	61%

Reading the numbers. The model has fully internalised the *schema*: every prediction is well-formed JSON in the right shape (100%). The harder skill — emitting a construction that is *geometrically executable* — succeeds outright 61% of the time. The remaining 39% fail for diagnosable, recurring reasons rather than malformed output, which is exactly what the repair loop targets.

6.1 Failure taxonomy

The 12 failures on the held-out set group into three causes:

cause	**example error**	**share**
undefined / mis-cased name or bad command	`Line_B_C` used but never defined; `tangent(...)` lowercase; `Midpoint(line)` on a line	majority
point at infinity (`nan`)	parallel-line intersection ⇒ `(nan, nan)`	several
degenerate collapse	too many coincident points (e.g. "3 distinct of 9")	a few

These are precisely the error classes the repair instruction addresses ("define every name before use; exact GeoGebra casing; avoid points at infinity"), so the 61% figure is a *lower bound* on what the full generate–validate–repair system delivers; `repair_loop.py` exists to recover the recoverable cases by re-prompting with the exact error.

6.2 What the result demonstrates

- A 3B model, QLoRA-tuned on only 201 verified examples, learns the target DSL well enough to produce a directly-renderable Olympiad figure for the majority of unseen problems.
- Schema adherence is effectively solved (100%), so the residual error is purely *geometric/semantic* — the tractable part, addressable by the in-the-loop verifier.
- The oracle-in-the-loop design means a modest raw accuracy is amplified by automatic validation and repair into a usable end-to-end tool.

7 Limitations and honest caveats

- **Renderability ≠ truth at inference.** The production `post_filter` checks that a figure draws and is non-degenerate, not that it proves the stated theorem. The numeric `checks`

oracle closes this gap but needs a ground-truth assertion, which unseen problems lack. (One shipped prediction renders yet pairs the wrong intersection points — valid figure, wrong geometry.)

- **Repository snapshots vs. code.** The shipped `generated_problems.jsonl` is renumbered 1–150 with no `checks`, whereas the generation code assumes ids ≥ 1000 with semantic checks; one training docstring still says "7B" though the config trains 3B. These are documentation/snapshot mismatches, not logic errors.
- **Small held-out set.** 31 problems give a clear signal but a wide confidence interval; the 61% figure should be read as indicative.
- **Model weights excluded.** The ∼2 GB GGUF is gitignored, so the repository is reproducible from scripts but does not embed the trained model.

8 Reproduction summary

1. **Validate the seed:** `python validate.py` (82/82 clean).
2. **Grow the corpus:** `python datagen/generate.py -n N` (API) or `validate_candidates.py` (no API); audit with `verify_dataset.py`.
3. **Prepare data:** `python finetune/prepare_data.py` → train/eval split.
4. **Train (GPU box):** `python finetune/qlora_train.py` → LoRA adapters.
5. **Predict & repair:** `infer.py`, `repair_loop.py`.
6. **Score (where GeoGebra runs):** `python finetune/eval_predictions.py -trials 7`.
7. **Serve locally:** `export_gguf.py` → Ollama (`geom.Modelfile`) → `predict_local.py` → paste JSON into `renderer.html`.

All quantitative results in this report were produced by re-running the project's own validation harness (`validate.py`, `datagen/verify_dataset.py`, `finetune/eval_predictions.py`) on the committed data and predictions.